

## Architecture

Goal: provide Angular support, while reusing common Content SDK concepts like `scClient`, `component-map`, `import-map` (later), `scConfig` (CLI config too).

Challenges:

- [Angular POC | The Challenge](#)
- all imported logic getting into angular bundle (server and client)
- `process.env` not being available in client

Angular implementation rests on the loader system described in POC doc ([Angular POC | LoaderFn \(Loader Function\)](#) )

## Loaders

Loaders are implemented as Angular route data resolvers: [Route data resolvers • Angular](#) . They populate the `page` , `dictionary` and potentially other route props when a request is processed by Angular SSR.

Loaders are configured on a route like this:

```
1 {  
2     path: '**',  
3     component: PageComponent,  
4     resolve: {  
5         page: loaderResolver('page'),  
6         dictionary: loaderResolver('dictionary'),  
7     },  
8 }
```

Loaders are retrieved from loader registry, injected in `app.config.ts`:

```
1 import { LOADERS } from '../content-sdk/loaders';  
2 ...  
3 providers: [  
4     ...  
5     provideLoaderRegistry(LOADERS),  
6     ...  
7 ]
```

and the loader implementation lives within the app. For example, the default `page` loader:

```
1 import type { LoaderFn, Page } from '@sitecore-content-sdk/angular';  
2 import { NotFoundNavigationError, resolveSitecorePage } from  
3 '@sitecore-content-sdk/angular';  
4 import scConfig from '../../sitecore.config';  
5 import { getClient } from '../client/sitecore-client';  
6 /**  
7  * Page loader: fetches layout data from Sitecore for the current URL.
```

```

8  * Uses imported config and {@link getClient} so this runs outside
   Angular injection context.
9  */
10 export const pageLoader: LoaderFn<Page> = async (context) => {
11   const page = await resolveSitecorePage(context.url, scConfig,
   getClient());
12   if (!page) {
13     throw new NotFoundNavigationError();
14   }
15   return page;
16 };
17

```

**loaderResolver** is the main entry point for loaders logic.

- On server
  - retrieves loaders from LOADER\_REGISTRY by name
  - executes loaders
  - writes loader result to TRANSFER\_STATE so that its value can be used faster on client navigation
- On browser
  - attempts to read loader result from TRANSFER\_STATE
  - if absent, invokes a request to loader-data-service expressJS middleware, using the loader-data.service
    - request promise is written to a **pending** collection
    - if another request is trying to execute the same loader for the same route, a pending promise resolution will be used instead of launching a new express JS request (for performance reasons)
    - express middleware retrieves a loader from registry, executes it and responds with a loader result
- Depending on the loader result (data, error, notFound), either a route prop is populated, or loaderResolver invokes a redirect to error/notFound page

#### PreLoaderDataService

Each router request has its own little sidequest in the form of pre-loader-data-service. On browser routing, the service subscribes to Angular's **ActivationStart** event and invokes all loaders for a route in parallel, as a pre-warm measure. Angular data resolvers are executed sequentially - and this mechanism allows us to speed this up.

#### Loaders logic considerations

Since loaders can be executed from **loaderResolver** directly or from the express JS middleware, they cannot use Angular's `inject()` mechanism. This is a known limitation - and things like `config` or `scClient` should be imported instead.



**Could not identify user**

## Config and env

Angular reuses the common `defineConfig` logic. One difference - `process.env` is not available on client side, and attempts to import a `defineConfig` result throws an exception.

To overcome this, we execute a build-time script that:

- Reads `CSDK_PUBLIC_...` env variables from server `process.env`
- writes them to `environment.dev/prod.ts` files, depending on the command launched (npm run dev / npm run start)
- Angular bundler builds their content into runtime `environment.ts` file. Contents of the file are passed to `defineConfig` - as opposed to using `process.env`, which is undefined in Angular browser context

## Components

All components are standalone according to Angular best practices. Content SDK component map only supports standalone components too.

Component map structure and logic follows next.js implementation:

- PascalCase for component names and imports for compatibility with component names in SAI
- Imports with `Default` and variants for each component
- Same variant logic for multi-file separation: `file1.default.ts` and `file1.var.ts` will be considered variants
- Some functions are now reused between next.js and angular component map generation.
- Angular placeholder uses the same component map format and high level resolution logic as next.js
- Component map generation is configured through `sitecore.cli.config.ts` file

## SSR

Angular app registers the loader-data-service middleware in `server.ts` before registering browser bundle and the main Angular SSR bundle.

## Config

Angular reuses the common Content SDK `sitecore.config.ts` approach to configuration.

## Fields and Directives

TBA

**Editing**

TBA

**Multisite**

TBA

**Personalization**

TBA